

OpenAI-compatible

Cache · Failover · Budgets

CUSTOMER GUIDE

The Conductor

A transparent proxy for the OpenAI API that adds a two-layer cache, multi-provider failover, per-key budget enforcement, and a live observability dashboard — with a one-line integration.

Live demo: <https://conductor-demo.fly.dev>

Prepared: July 13, 2026

Audience: Customers integrating against the gateway (demo key or self-hosted)

Contents

1. What The Conductor Does

2. Get Started in 60 Seconds (Live Demo)

3. How the Cache Works — And Why Your Bill Drops

4. Failover — What Happens When a Provider Goes Down

5. Budgets — Hard Caps on Spend, Per Key

6. The Observability Dashboard

7. Self-Hosting Your Own Instance

8. Security Model

9. Known Limits & What's Out of Scope

10. Troubleshooting

1. What The Conductor Does

The Conductor sits between your application and the LLM providers you already call. It speaks the exact same request/response shape as the OpenAI Chat Completions API, so integrating it is a one-line change: point your existing SDK's `base_url` at the gateway instead of at OpenAI directly. Everything downstream — your prompts, your streaming handling, your error handling — stays the same.

In front of your provider traffic, the gateway adds four things you'd otherwise have to build yourself:

Capability	What it means for you as a customer
Two-layer cache	Identical and near-identical prompts are served from cache instead of re-billed to the provider. Exact match via Redis, near-duplicate match via a Postgres/pgvector semantic index.
Multi-provider failover	If your primary provider (e.g. OpenAI) returns an error before any output is sent, the gateway automatically retries against a fallback provider (e.g. Anthropic) — your request still succeeds.
Per-key budget enforcement	Every API key has a hard spend cap. Once it's hit, the gateway rejects further requests with a 402 rather than letting a bug or a runaway loop rack up an unbounded bill.
Live observability dashboard	A read-only web dashboard shows spend, cache hit rate, latency, failover events, and per-key usage — no instrumentation required on your side.

WHY THIS MATTERS

You get provider redundancy, cost control, and visibility without changing how you call the API or maintaining any of that infrastructure yourself.

2. Get Started in 60 Seconds (Live Demo)

The fastest way to try The Conductor is against the hosted demo instance. No signup, no install.

1 Point your OpenAI SDK at the gateway

```
# Python
import openai
client = openai.OpenAI(
    base_url="https://conductor-demo.fly.dev/v1",
    api_key="demo-key"
)
resp = client.chat.completions.create(
    model="fast",
    messages=[{"role": "user", "content": "hello"}]
)
print(resp.choices[0].message.content)
```

2 Or call it directly with curl

```
curl https://conductor-demo.fly.dev/v1/chat/completions \
-H "Authorization: Bearer demo-key" \
-H "Content-Type: application/json" \
-d '{"model":"fast","messages":[{"role":"user","content":"hello"}]}'
```

3 Check that it's alive

```
curl https://conductor-demo.fly.dev/health
# → {"status":"ok","db":true,"redis":true}
```

SHARED BUDGET — READ BEFORE YOU LOAD-TEST

`demo-key` is a real, shared key hitting real OpenAI/Anthropic accounts. It is hard-capped at **\$10/month** by the gateway's own budget enforcement — once the cap is hit, every request gets a 402 until the monthly reset, regardless of who's calling. It resets monthly. Don't assume it has budget left if others have used it recently.

3. How the Cache Works — And Why Your Bill Drops

Every completion request is checked against two cache layers, in order, before it ever reaches a provider:

Layer	Store	Match type	Behavior
1 — Exact	Redis	SHA-256 hash of the normalized request	Byte-identical requests return instantly from cache.
2 — Semantic	Postgres + pgvector	Cosine similarity (HNSW index) against an embedding of the prompt	Near-duplicate phrasing above a similarity threshold (default <code>0.95</code>) returns a previously cached response.

The cache entry is written once the full response stream has completed — this keeps every cached reply, streamed or not, byte-identical to what a real call would have produced.

Opting out per request

If a request must always hit a live provider (e.g. anything with real-time or non-deterministic requirements), add a cache override to the request body:

```
{
  "model": "fast",
  "messages": [...],
  "cache": {"no_cache": true}
}
```

GOOD TO KNOW

Near-duplicate matches can occasionally replay a response from a different provider than the one currently live — that's expected cache behavior, not a routing bug. If you need to distinguish requests that only differ by a number or ID (e.g. two prompts that differ only by "id: 41" vs "id: 42"), be aware the default 0.95 threshold does not guarantee separating those — see Section 9.

4. Failover — What Happens When a Provider Goes Down

Each model alias you call (e.g. `fast`) resolves to an ordered chain of providers. If the primary provider's response indicates a retryable failure — and this check happens **before any output token is streamed back to you** — the gateway automatically retries the next provider in the chain.

What you'll see

- A successful response, indistinguishable in shape from a primary-provider response
- Slightly higher latency (one backoff + a second provider round trip)
- A recorded event visible in the dashboard's Failovers view

What's deliberately not supported

- **Mid-stream failover.** Once your client has partial streamed output, the gateway will not switch providers mid-response — that would produce a corrupted, seamed reply. If a provider fails after streaming has started, the failure surfaces to you as a stream error.

DESIGN RATIONALE

This is an intentional, documented tradeoff (ADR-002): pretending mid-stream failover is clean would be dishonest. Only pre-first-token failures are retried.

5. Budgets — Hard Caps on Spend, Per Key

Every API key has a soft-warn threshold and a hard limit, tracked in cents, reset monthly.

Threshold	Behavior
Soft limit	Requests still succeed; spend is logged and visible in the dashboard.
Hard limit	Every subsequent request on that key is rejected with HTTP <code>402 Payment Required</code> until the monthly reset — no exceptions, no grace requests.

Spend accounting happens on every authenticated request via an atomic increment (`INCRBYFLOAT` in Redis), so concurrent requests can't race past the cap.

FOR YOUR OWN KEY

If you're self-hosting, budgets are configured per-key at the database level (see `gateway/db` migrations). On the live demo, `demo-key` 's cap is fixed at \$10/month and cannot be raised — request your own key or self-host if you need a higher ceiling.

6. The Observability Dashboard

Six read-only endpoints under `/v1/observability/*` back a Next.js dashboard showing real-time gateway activity — no logging or instrumentation needed on your end.

Endpoint	Shows
<code>GET /v1/observability/spend</code>	Spend over time, bucketed by window
<code>GET /v1/observability/cache</code>	Cache hit rate (exact vs. semantic) over a time window
<code>GET /v1/observability/latency</code>	Latency distribution, including cache-hit vs. cache-miss comparison
<code>GET /v1/observability/savings</code>	Estimated cost saved by cache hits
<code>GET /v1/observability/failovers</code>	Failover events — which alias, which fallback depth, when
<code>GET /v1/observability/keys</code>	Per-key usage and remaining budget

All six endpoints accept a `?window=` query parameter (e.g. `7d`) to control the reporting range.

AUTH REQUIRED

These endpoints sit behind a separate static bearer token (`Authorization: Bearer <DASHBOARD_AUTH_TOKEN>`), distinct from your API key. In a self-hosted production deployment this token is required at startup; in development it's optional (an unset token just logs a warning).

7. Self-Hosting Your Own Instance

If you want your own instance — your own budget ceiling, your own keys, full control — you have two paths.

Option A — Local, via Docker Compose

```
OPENAI_API_KEY=sk-... ANTHROPIC_API_KEY=sk-... \
docker compose -f infra/docker-compose.yml up
```

This brings up the gateway, Postgres (with pgvector), and Redis, all locally, with no shared budget and no external dependency on Neon/Upstash.

Option B — Deploy your own Fly.io instance

The full runbook lives in `gateway/DEPLOY.md`. In short:

1. Install `flyctl`, log in, and add a payment method to your Fly org (required even on the free allowance).
2. Create a free Neon Postgres project (pgvector is built in — do not use Fly's stock Postgres, it lacks pgvector).
3. Create a free Upstash Redis database.
4. Run `fly launch --no-deploy --copy-config`, then `fly secrets set` with your `DATABASE_URL`, `REDIS_URL`, `OPENAI_API_KEY`, `ANTHROPIC_API_KEY` — all in a single shell invocation (see the callout below).
5. Run `fly deploy`. Migrations apply automatically on boot, including seed API keys.

COMMON SELF-HOST MISTAKE
`fly secrets set` must receive all your exported variables in the *same* shell invocation as the export. Splitting the export and the `secrets set` across separate shell calls silently sends empty strings for every secret — the deployed machine then crash-loops trying to connect to `127.0.0.1:5432`.

Key environment variables

Variable	Purpose
<code>DATABASE_URL</code>	Postgres connection string (must support pgvector)
<code>REDIS_URL</code>	Redis connection string
<code>OPENAI_API_KEY</code> / <code>ANTHROPIC_API_KEY</code>	Your real provider credentials — referenced by the DB, never stored in it
<code>SEMANTIC_SIMILARITY_THRESHOLD</code>	Cosine-similarity cutoff for semantic cache hits (default <code>0.95</code>)

ENVIRONMENT	production requires DASHBOARD_AUTH_TOKEN to be set, or startup fails
DASHBOARD_AUTH_TOKEN	Static bearer token guarding the observability endpoints

8. Security Model

- **Chat completions** are authenticated per-key via `Authorization: Bearer <key>`, resolved and budget-checked against the database on every request.
- **Observability endpoints** use a single static bearer token, separate from API keys.
- Neither mechanism is a substitute for real user accounts. For an enterprise deployment, put the gateway behind a reverse proxy or SSO layer (nginx, an identity-aware proxy, or Fly.io app-level access controls), and terminate TLS in front of both the gateway and dashboard so bearer tokens never travel in the clear.
- Provider secrets are referenced via environment variables and never stored in the database.

9. Known Limits & What's Out of Scope

Limitation	Detail
Mid-stream failover	Not supported by design — see Section 4.
Two providers only	OpenAI and Anthropic. A third provider is a mechanical addition (a new translation adapter) but is intentionally out of scope to keep the demonstrated pattern focused.
Semantic threshold has a known gap	The default 0.95 threshold is the safest single value found against a synthetic eval set, but does not fully meet a $\leq 1\%$ false-positive target — a numeric-ID near-miss can out-score a true duplicate. If your traffic frequently varies only by IDs or numbers in an otherwise identical prompt, treat semantic cache hits on those requests with caution, or disable caching for them with <code>"cache": {"no_cache": true}</code> .
No user accounts / RBAC	Per-key and dashboard-token auth stop anonymous access but don't provide multi-user accounts or role-based access control — that's enterprise-tier scope.
Horizontal scale untested	Multiple gateway workers/machines work correctly (atomic spend counters, per-process DB connections) but haven't been load-tested at that scale.

10. Troubleshooting

Symptom	Likely cause	Fix
<code>402 Payment Required</code>	Your key has hit its hard spend limit for the month	Wait for the monthly reset, or use a key with a higher cap
Identical requests return suspiciously fast, same content every time	Exact cache hit	Expected behavior — add <code>"cache": {"no_cache": true}</code> if you need a live call
Response seems to come from the "wrong" provider given current outages	Semantic cache replayed a cached response from before the outage	Not a routing bug — disable cache for that request to confirm live routing
Self-hosted machine crash-loops connecting to <code>127.0.0.1:5432</code>	Secrets were set with empty values (see Section 7 callout)	Re-run <code>fly secrets set</code> with the export in the same shell invocation
<code>fly launch</code> fails with a machine-count/quota error	No payment method on the Fly org	Add a card under Fly dashboard → org → Billing
Migration fails creating the vector extension	Using Fly's stock Postgres instead of Neon	Use Neon (or any Postgres with pgvector built in) — not <code>fly postgres create</code>